

# LARAVEL PART 4

IT9a/L



# Databases, Migrations, and Eloquent



# Databases, Migrations, and Eloquent

- Under the database folder, you can see the ***database.sqlite*** (the default database of the project)
- Go to the **.env** file and configure the database. (we will use the **MySQL**)
- Create a database and put the name of your database to the **DB\_DATABASE**.
- **.env** – is a plain-text configuration file used to store sensitive information and environment-specific settings. Instead of hardcoding things like your database password or API keys directly into your code, you store them here.



# Databases, Migrations, and Eloquent

- Type the **php artisan** into the terminal and you can see the list of commands for the artisan.
- **Artisan** – is the ***Command-Line Interface (CLI)*** that comes packaged with every Laravel project. Instead of manually creating files, writing boilerplate code, or clicking through database managers, you give **Artisan** a text command in your terminal, and it does the heavy lifting for you.
- Create notes table using the **php artisan make:migration** and named it ***create\_notes\_table***
- The name of the migration should be the description of what that migration should do.



# Databases, Migrations, and Eloquent

- The **up** and **down methods** – The **up()** method is executed when you run the command *php artisan migrate*. Its job is to add something to your database. The **down()** method is the exact opposite. It is executed when you run *php artisan migrate:rollback*. Its job is to reverse whatever the up() method did.
- add column in the up() method-> `$table->text('description')`
- to add the actual table to the database, run the command -> ***php artisan migrate***
- After running the command, it will now store the table to the database
- Try to add fields to the notes table in the database(phpmyadmin)



# Databases, Migrations, and Eloquent

## Utilize the database table

### 1. Using Generic Database Query

- By using the DB façade, you can get the data from the notes table in the database -> **`$notes = DB::table('notes')->get();`**
- Try the **`dd($notes)`** to see if it has the added fields (Laravel collection objects)
- Use the **`return $notes;`** --> Laravel is smart enough to convert it to json.
- Try the **`return $notes[0]`** and **`$notes[0]->description.`**
- When you go back to UI, it will cause an error.



# Databases, Migrations, and Eloquent

- Update the ***\$note*** to ***\$note->description*** in the views --> the \$note is now an object that is a representation of one row in our table.
- We try to add the status column to our table. There are 2 ways to do it.
- 1<sup>st</sup>, add new column in the create notes table -> ***\$table->string('status');*** then run the ***php artisan migrate:refresh.***
- This will roll back your migration and rerun the migration from scratch. You're going to lose all your data.



# Databases, Migrations, and Eloquent

- 2<sup>nd</sup>, create another migration -> ***php artisan make:migration add\_status\_to\_notes\_table***
- In the **up()** method, add the **`$table->string('status');`**
- In the **down()** method, add **`$table->dropColumn('status');`**
- Run the ***php artisan migrate***.
  
- back to the ***web.php***, you can use the **where()** method to filter out the result by status.





# Databases, Migrations, and Eloquent

## 2. Using Eloquent ORM

- **Laravel's Eloquent ORM** – Eloquent is Laravel's built-in **Object-Relational Mapper (ORM)**. It provides a beautiful, simple "Active Record" implementation for working with your database. Instead of writing raw SQL queries, Eloquent allows you to interact with your database tables using PHP objects called **Models**. It acts as an intuitive translator between your PHP code and your relational database.
- What if we had the code `$note = Note::find(1);` (Note is an eloquent model)



# Databases, Migrations, and Eloquent

- Use the ***php artisan make:model*** command in the terminal to create a model.
- Name the model: Note
- Go to **app/models**
- You can see the **Note** model and it is empty but you can have a lot of functionality even with empty class.
- Go to the **web.php** and import the Note model -> ***use App/Models/Note;***
- You can now use the ***Note::find(1);***
- return the **\$note** and it will be converted to Json
- if we want to fetch all the notes, then use the **\$notes = Note::all();**



# Databases, Migrations, and Eloquent

## Creating New Note

- Use the Note model
- `Note::create([  
'description' => $note,  
'status' => 'pending'])`
- Try create another user input then you can see that it is working. Check the table if it adds a new field.



# Databases, Migrations, and Eloquent

## Filter Results by status

- What if some status was set to complete, and you only care about the pending ones.
- Add where condition -> **`$notes = Note::where('status', 'pending')->get();`**
- You can't use **`all()`** it is because you add conditions, use the **`get()`** instead.



# Databases, Migrations, and Eloquent

## Filter Results by status

- We can also use **query string filtering**.
- `$note = Note::query()->when(request('status'), function($query, $status){  
dd($status);  
}) ->get();`
- Change the ***dd(\$status);*** to ***\$query->where('status', \$status);***